

BAAI / **bge-m3**



2.4k



Beijing Academy of Arti...

2.7k



Sentence Similarity



sentence-transformers



PyTorch



ONNX

xlm-roberta

feature-extraction



text-embeddings-inference



arxiv:5 papers



License: mit



Train



Deploy



Use this model

Model card

Files

xet



Community

130

Downloads last month

5,912,027



Inference Providers

NEW

HF

Sentence Similarity

Examples



Source Sentence

Your sentence here...

Sentences to compare to

Your sentence here...

Your sentence here...

Add Sentence

Generate

</> View Code

Run 15,000+ Models Instantly

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

BAAI/bge-m3 is supported by the following Inference Providers:



HF Inference API

</> View API Code

Dismiss



Maximize

Model tree for BAAI/bge-m3

Adapters

3 models

Finetunes

346 models

Merges

1 model

Quantizations

64 models

Spaces using BAAI/bge-m3 100

Collection including BAAI/bge-m3

BGE Collection

31 items • Updated 19 days ago • 136

For more details please refer to our github repo: <https://github.com/FlagOpen/FlagEmbedding>

🔗 BGE-M3 ([paper](#), [code](#))

In this project, we introduce BGE-M3, which is distinguished for its versatility in Multi-Functionality, Multi-Linguality, and Multi-Granularity.

- Multi-Functionality: It can simultaneously perform the three common retrieval functionalities of embedding model: dense retrieval, multi-vector retrieval, and sparse retrieval.
- Multi-Linguality: It can support more than 100 working languages.
- Multi-Granularity: It is able to process inputs of different granularities, spanning from short sentences to long documents of up to 8192 tokens.

Some suggestions for retrieval pipeline in RAG

We recommend to use the following pipeline: hybrid retrieval + re-ranking.

- Hybrid retrieval leverages the strengths of various methods, offering higher accuracy and stronger generalization capabilities. A classic example: using both embedding retrieval and the BM25 algorithm. Now, you can try to use BGE-M3, which supports both embedding and sparse retrieval. This allows you to obtain token weights (similar to the BM25) without any additional cost when generate dense embeddings. To use hybrid retrieval, you can refer to [Vespa](#) and [Milvus](#).
- As cross-encoder models, re-ranker demonstrates higher accuracy than bi-encoder embedding model. Utilizing the re-ranking model (e.g., [bge-reranker](#), [bge-reranker-v2](#)) after retrieval can further filter the selected text.

🔗 News:

- 2024/7/1: **We update the MIRACL evaluation results of BGE-M3.** To reproduce the new results, you can refer to: [bge-m3_miracl_2cr](#). We have also updated our [paper](#) on arXiv.
▶ Details
- 2024/3/20: **Thanks Milvus team!** Now you can use hybrid retrieval of bge-m3 in Milvus: [pymilvus/examples/hello_hybrid_sparse_dense.py](#).
- 2024/3/8: **Thanks for the [experimental results](#) from @Yannael.** In this benchmark, BGE-M3 achieves top performance in both English and other languages, surpassing models such as OpenAI.
- 2024/3/2: Release unified fine-tuning [example](#) and [data](#)
- 2024/2/6: We release the [MLDR](#) (a long document retrieval dataset covering 13 languages) and [evaluation pipeline](#).
- 2024/2/1: **Thanks for the excellent tool from Vespa.** You can easily use multiple modes of BGE-M3 following this [notebook](#)

Specs

- Model

Model Name	Dimension	Sequence Length	Introduction
BAAI/bge-m3	1024	8192	multilingual; unified fine-tuning (dense, sparse, and colbert) from bge-m3-unsupervised
BAAI/bge-m3-unsupervised	1024	8192	multilingual; contrastive learning from bge-m3-retromae
BAAI/bge-m3-retromae	--	8192	multilingual; extend the max_length of xlm-roberta to 8192 and further pretrained via retromae
BAAI/bge-large-en-v1.5	1024	512	English model
BAAI/bge-base-en-v1.5	768	512	English model
BAAI/bge-small-en-v1.5	384	512	English model

- Data

Dataset	Introduction
MLDR	Docuemtn Retrieval Dataset, covering 13 languages
bge-m3-data	Fine-tuning data used by bge-m3

FAQ

1. Introduction for different retrieval methods

- Dense retrieval: map the text into a single embedding, e.g., [DPR](#), [BGE-v1.5](#)
- Sparse retrieval (lexical matching): a vector of size equal to the vocabulary, with the majority of positions set to zero, calculating a weight only for tokens present in the text. e.g., [BM25](#), [unicoil](#), and [splade](#)
- Multi-vector retrieval: use multiple vectors to represent a text, e.g., [ColBERT](#).

2. How to use BGE-M3 in other projects?

For embedding retrieval, you can employ the BGE-M3 model using the same approach as BGE. The only difference is that the BGE-M3 model no longer requires adding instructions to the queries.

For hybrid retrieval, you can use [Vespa](#) and [Milvus](#).

3. How to fine-tune bge-M3 model?

You can follow the common in this [example](#) to fine-tune the dense embedding.

If you want to fine-tune all embedding function of m3 (dense, sparse and colbert), you can refer to the [unified fine-tuning example](#)

[🔗 Usage](#)

Install:

```
git clone https://github.com/FlagOpen/FlagEmbedding.git
cd FlagEmbedding
pip install -e .
```

or:

```
pip install -U FlagEmbedding
```

[🔗 Generate Embedding for text](#)

- Dense Embedding

```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3',
                        use_fp16=True) # Setting use_fp16 to True speeds up computation with a slight loss in accuracy

sentences_1 = ["What is BGE M3?", "Defination of BM25"]
sentences_2 = ["BGE M3 is an embedding model supporting dense retrieval, lexical matching and mul",
               "BM25 is a bag-of-words retrieval function that ranks a set of documents based on"]

embeddings_1 = model.encode(sentences_1,
                             batch_size=12,
                             max_length=8192, # If you don't need such a long length, you can set a smaller value
                             )[ 'dense_vecs' ]
embeddings_2 = model.encode(sentences_2)[ 'dense_vecs' ]
similarity = embeddings_1 @ embeddings_2.T
```

```
print(similarity)
# [[0.6265, 0.3477], [0.3499, 0.678 ]]
```

You also can use sentence-transformers and huggingface transformers to generate dense embeddings. Refer to [baai_general_embedding](#) for details.

- Sparse Embedding (Lexical Weight)

```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True) # Setting use_fp16 to True speeds up computation

sentences_1 = ["What is BGE M3?", "Defination of BM25"]
sentences_2 = ["BGE M3 is an embedding model supporting dense retrieval, lexical matching and multi-vector search",
               "BM25 is a bag-of-words retrieval function that ranks a set of documents based on their relevance to a query"]

output_1 = model.encode(sentences_1, return_dense=True, return_sparse=True, return_colbert_vecs=False)
output_2 = model.encode(sentences_2, return_dense=True, return_sparse=True, return_colbert_vecs=False)

# you can see the weight for each token:
print(model.convert_id_to_token(output_1['lexical_weights']))
# [{'What': 0.08356, 'is': 0.0814, 'B': 0.1296, 'GE': 0.252, 'M': 0.1702, '3': 0.2695, '?': 0.046},
#  {'De': 0.05005, 'fin': 0.1368, 'ation': 0.04498, 'of': 0.0633, 'BM': 0.2515, '25': 0.3335}]

# compute the scores via lexical mathcing
lexical_scores = model.compute_lexical_matching_score(output_1['lexical_weights'][0], output_2['lexical_weights'])
print(lexical_scores)
# 0.19554901123046875

print(model.compute_lexical_matching_score(output_1['lexical_weights'][0], output_1['lexical_weights']))
# 0.0
```

- Multi-Vector (ColBERT)

```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)

sentences_1 = ["What is BGE M3?", "Defination of BM25"]
sentences_2 = ["BGE M3 is an embedding model supporting dense retrieval, lexical matching and multi-vector search",
               "BM25 is a bag-of-words retrieval function that ranks a set of documents based on their relevance to a query"]
```

```

output_1 = model.encode(sentences_1, return_dense=True, return_sparse=True, return_colbert_vecs=True)
output_2 = model.encode(sentences_2, return_dense=True, return_sparse=True, return_colbert_vecs=True)

print(model.colbert_score(output_1['colbert_vecs'][0], output_2['colbert_vecs'][0]))
print(model.colbert_score(output_1['colbert_vecs'][0], output_2['colbert_vecs'][1]))
# 0.7797
# 0.4620

```

🔗 Compute score for text pairs

Input a list of text pairs, you can get the scores computed by different methods.

```

from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)

sentences_1 = ["What is BGE M3?", "Defination of BM25"]
sentences_2 = ["BGE M3 is an embedding model supporting dense retrieval, lexical matching and mul",
               "BM25 is a bag-of-words retrieval function that ranks a set of documents based on"]

sentence_pairs = [[i,j] for i in sentences_1 for j in sentences_2]

print(model.compute_score(sentence_pairs,
                          max_passage_length=128, # a smaller max length leads to a lower latency
                          weights_for_different_modes=[0.4, 0.2, 0.4])) # weights_for_different_m

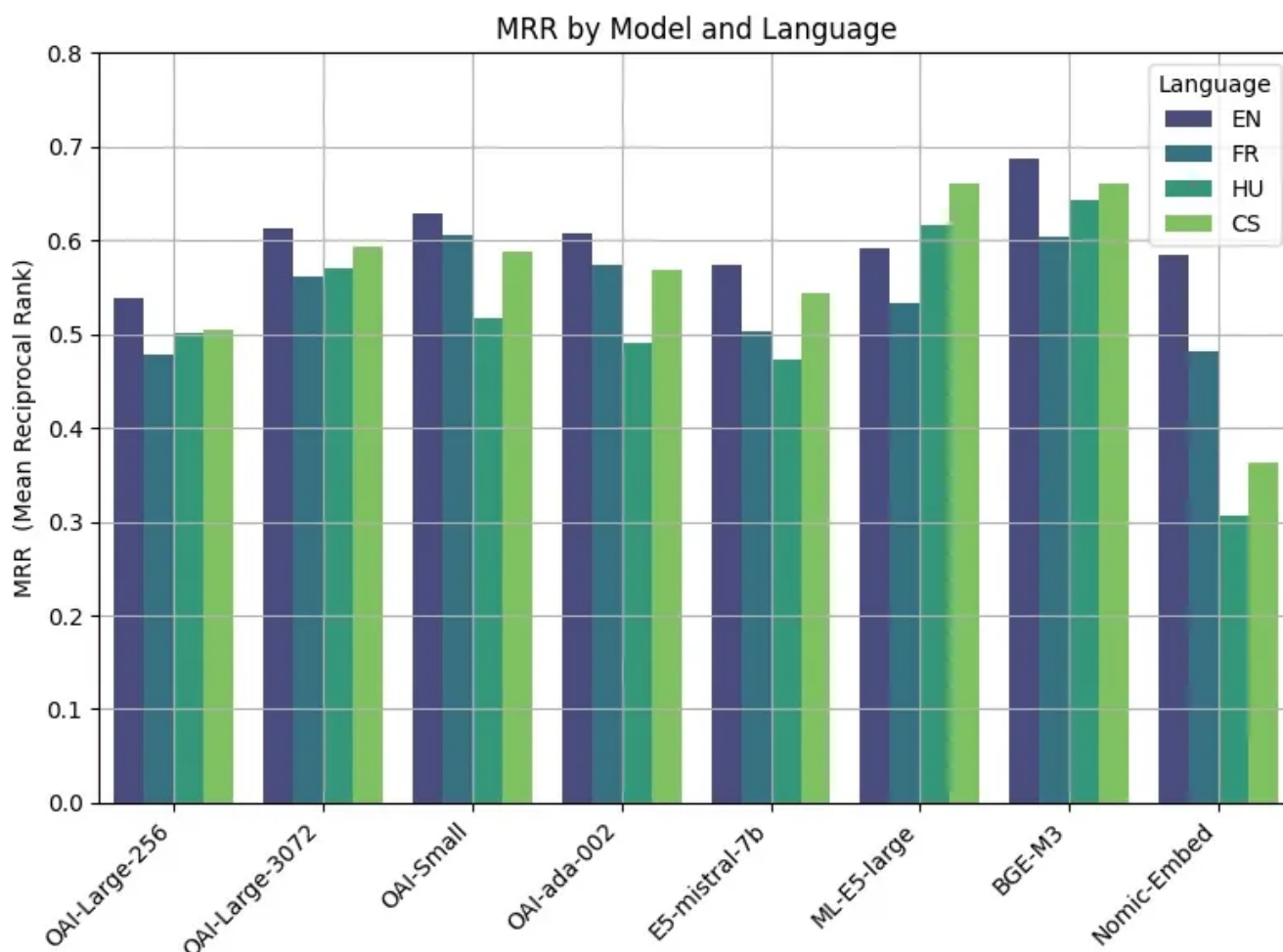
# {
#   'colbert': [0.7796499729156494, 0.4621465802192688, 0.4523794651031494, 0.7898575067520142],
#   'sparse': [0.195556640625, 0.00879669189453125, 0.0, 0.1802978515625],
#   'dense': [0.6259765625, 0.347412109375, 0.349853515625, 0.67822265625],
#   'sparse+dense': [0.482503205537796, 0.23454029858112335, 0.2332356721162796, 0.51224774122238],
#   'colbert+sparse+dense': [0.6013619303703308, 0.3255828022956848, 0.32089319825172424, 0.62329]
# }

```

🔗 Evaluation

We provide the evaluation script for [MKQA](#) and [MLDR](#)

🔗 Benchmarks from the open-source community



The BGE-M3 model emerged as the top performer on this benchmark (OAI is short for OpenAI). For more details, please refer to the [article](#) and [Github Repo](#)

Our results

- Multilingual (Miracl dataset)

Model	Avg	ar	bn	en	es	fa	fi	fr	hi	id	ja	ko	ru	sw	te	th	zh	de	yo
Baselines (<i>Prior Work</i>)																			
BM25	31.9	39.5	48.2	26.7	7.7	28.7	45.8	11.5	35.0	29.7	31.2	37.1	25.6	35.1	38.3	49.1	17.5	12.0	56.1
mDPR	41.8	49.9	44.3	39.4	47.8	48.0	47.2	43.5	38.3	27.2	43.9	41.9	40.7	29.9	35.6	35.8	51.2	49.0	39.6
mContriever	43.1	52.5	50.1	36.4	41.8	21.5	60.2	31.4	28.6	39.2	42.4	48.3	39.1	56.0	52.8	51.7	41.0	40.8	41.5
mE5 _{large}	66.6	76.0	75.9	52.9	52.9	59.0	77.8	54.5	62.0	52.9	70.6	66.5	67.4	74.9	84.6	80.2	56.0	56.4	78.3
E5 _{mistral-7b}	63.4	73.3	70.3	57.3	52.2	52.1	74.7	55.2	52.1	52.7	66.8	61.8	67.7	68.4	73.9	74.0	54.0	54.1	79.7
OpenAI-3	54.9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
M3-Embedding (<i>Our Work</i>)																			
Dense	69.2	78.4	80.0	56.9	56.1	60.9	78.6	58.3	59.5	56.1	72.8	69.9	70.1	78.7	86.2	82.6	62.7	56.7	81.8
Sparse	53.9	67.1	68.9	43.8	38.6	45.1	65.4	35.3	48.2	48.9	56.1	61.5	44.5	57.9	79.1	70.9	36.1	32.5	70.0
Multi-vec	70.5	79.6	81.0	59.3	57.8	62.0	80.1	59.4	61.5	58.3	74.5	71.2	71.2	79.1	87.9	83.0	63.7	58.0	82.4
Dense+Sparse	70.4	79.6	80.7	58.8	58.1	62.3	79.7	58.0	62.9	58.3	73.9	71.2	69.8	78.5	87.2	83.1	63.5	57.7	83.3
All	71.5	80.2	81.5	59.6	59.7	63.4	80.4	61.2	63.3	59.0	75.2	72.1	71.7	79.6	88.1	83.7	64.9	59.8	83.5

Table 1: **Multi-lingual retrieval performance on the MIRACL dev set** (measured by nDCG@10).

- Cross-lingual (MKQA dataset)

	Baselines (<i>Prior Work</i>)						M3-Embedding (<i>Our Work</i>)				
	BM25	mDPR	mContriever	mE5 _{large}	E5 _{mistral-7b}	OpenAI-3	Dense	Sparse	Multi-vec	Dense+Sparse	All
ar	18.9	48.2	58.2	68.7	59.6	65.6	71.1	23.5	71.4	71.1	71.5
da	49.3	67.4	73.9	77.4	77.8	73.6	77.2	55.4	77.5	77.4	77.6
de	35.4	65.8	71.7	76.9	77.0	73.6	76.2	43.3	76.3	76.4	76.3
es	43.4	66.8	72.6	76.4	77.4	73.9	76.4	50.6	76.6	76.7	76.9
fi	46.3	56.2	70.2	74.0	72.0	72.7	75.1	51.1	75.3	75.3	75.5
fr	45.3	68.2	72.8	75.5	78.0	74.1	76.2	53.9	76.4	76.6	76.6
he	26.9	49.7	63.8	69.6	47.2	58.1	72.4	31.1	72.9	72.5	73.0
hu	38.2	60.4	69.7	74.7	75.0	71.2	74.7	44.6	74.6	74.9	75.0
it	45.2	66.0	72.3	76.8	77.1	73.6	76.0	52.5	76.4	76.3	76.5
ja	24.5	60.3	64.8	71.5	65.1	71.9	75.0	31.3	75.1	75.0	75.2
km	27.8	29.5	26.8	28.1	34.3	33.9	68.6	30.1	69.1	68.8	69.2
ko	27.9	50.9	59.7	68.1	59.4	63.9	71.6	31.4	71.7	71.6	71.8
ms	55.9	65.5	74.1	76.3	77.2	73.3	77.2	62.4	77.4	77.4	77.4
nl	56.2	68.2	73.7	77.8	79.1	74.2	77.4	62.4	77.6	77.7	77.6
no	52.1	66.7	73.5	77.3	76.6	73.3	77.1	57.9	77.2	77.4	77.3
pl	40.8	63.3	71.6	76.7	77.1	72.7	76.3	46.1	76.5	76.3	76.6
pt	44.9	65.5	72.0	73.5	77.5	73.7	76.3	50.9	76.4	76.5	76.4
ru	33.2	62.7	69.8	76.8	75.5	72.0	76.2	36.9	76.4	76.2	76.5
sv	54.6	66.9	73.2	77.6	78.3	74.0	76.9	59.6	77.2	77.4	77.4
th	37.8	53.8	66.9	76.0	67.4	65.2	76.4	42.0	76.5	76.5	76.6
tr	45.8	59.1	71.1	74.3	73.0	71.8	75.6	51.8	75.9	76.0	76.0
vi	46.6	63.4	70.9	75.4	70.9	71.1	76.6	51.8	76.7	76.8	76.9
zh_cn	31.0	63.7	68.1	56.6	69.3	70.7	74.6	35.4	74.9	74.7	75.0
zh_hk	35.0	62.8	68.0	58.1	65.1	69.6	73.8	39.8	74.1	74.0	74.3
zh_tw	33.5	64.0	67.9	58.1	65.8	69.7	73.5	37.7	73.5	73.6	73.6
Avg	39.9	60.6	67.9	70.9	70.1	69.5	75.1	45.3	75.3	75.3	75.5

Table 3: Cross-lingual retrieval performance on MKQA (measured by Recall@100).

- Long Document Retrieval
 - MLDR:

	Max Length	Avg	ar	de	en	es	fr	hi	it	ja	ko	pt	ru	th	zh
<i>Baselines (Prior Work)</i>															
BM25	8192	53.6	45.1	52.6	57.0	78.0	75.7	43.7	70.9	36.2	25.7	82.6	61.3	33.6	34.6
mDPR	512	23.5	15.6	17.1	23.9	34.1	39.6	14.6	35.4	23.7	16.5	43.3	28.8	3.4	9.5
mContriever	512	31.0	25.4	24.2	28.7	44.6	50.3	17.2	43.2	27.3	23.6	56.6	37.7	9.0	15.3
mE5 _{large}	512	34.2	33.0	26.9	33.0	51.1	49.5	21.0	43.1	29.9	27.1	58.7	42.4	15.9	13.2
E5 _{mistral-7b}	8192	42.6	29.6	40.6	43.3	70.2	60.5	23.2	55.3	41.6	32.7	69.5	52.4	18.2	16.8
text-embedding-ada-002	8191	32.5	16.3	34.4	38.7	59.8	53.9	8.0	46.5	28.6	20.7	60.6	34.8	9.0	11.2
jina-embeddings-v2-base-en	8192	-	-	-	37.0	-	-	-	-	-	-	-	-	-	-
<i>M3-Embedding (Our Work)</i>															
Dense	8192	52.5	47.6	46.1	48.9	74.8	73.8	40.7	62.7	50.9	42.9	74.4	59.5	33.6	26.0
Sparse	8192	62.2	58.7	53.0	62.1	87.4	82.7	49.6	74.7	53.9	47.9	85.2	72.9	40.3	40.5
Multi-vec	8192	57.6	56.6	50.4	55.8	79.5	77.2	46.6	66.8	52.8	48.8	77.5	64.2	39.4	32.7
Dense+Sparse	8192	64.8	63.0	56.4	64.2	88.7	84.2	52.3	75.8	58.5	53.1	86.0	75.6	42.9	42.0
All	8192	65.0	64.7	57.9	63.8	86.8	83.9	52.2	75.5	60.1	55.7	85.4	73.8	44.7	40.0
<i>M3-w.o.long</i>															
Dense-w.o.long	8192	41.2	35.4	35.2	37.5	64.0	59.3	28.8	53.1	41.7	29.8	63.5	51.1	19.5	16.5
Dense-w.o.long (MCLS)	8192	45.0	37.9	43.3	41.2	67.7	64.6	32.0	55.8	43.4	33.1	67.8	52.8	27.2	18.2

Table 4: Evaluation of multilingual long-doc retrieval on the MLDR test set (measured by nDCG@10).

Please note that MLDR is a document retrieval dataset we constructed via LLM, covering 13 languages, including test set, validation set, and training set. We utilized the training set from MLDR to enhance the model's long document retrieval capabilities. Therefore, comparing baselines with Dense w.o.long (fine-tuning without long document dataset) is more equitable. Additionally, this long document retrieval dataset will be open-sourced to address the current lack of open-source multilingual long text retrieval datasets. We believe that this data will be helpful for the open-source community in training document retrieval models.

- NarritiveQA:

Model	Max Length	nDCG@10
Baselines (<i>Prior Work</i>)		
mDPR	512	16.3
mContriever	512	23.3
mE5 _{large}	512	24.2
bge-large-en-v1.5	512	27.3
E5 _{mistral-7b}	8192	49.9
text-embedding-ada-002	8191	41.1
text-embedding-3-large	8192	51.6
jina-embeddings-v2-base-en	8192	39.4
M3-Embedding (<i>Our Work</i>)		
Dense	8192	48.7
Sparse	8192	57.5
Multi-vec	8192	55.4
Dense+Sparse	8192	60.1
All	8192	61.7

Table 5: Evaluation on NarrativeQA (nDCG@10).

- Comparison with BM25

We utilized Pyserini to implement BM25, and the test results can be reproduced by this [script](#). We tested BM25 using two different tokenizers: one using Lucene Analyzer and the other using the same tokenizer as M3 (i.e., the tokenizer of xlm-roberta). The results indicate that BM25 remains a competitive baseline, especially in long document retrieval.

Method	Tokenizer	MIRACL	MKQA	MLDR
BM25	Analyzer	38.5	40.9	64.1
BM25	XLM-R	31.9	39.9	53.6
M3(Sparse)	XLM-R	53.9	45.3	62.2
M3(All)	XLM-R	71.5	75.5	65.0

Table 11: Comparison with the BM25 methods using different tokenizers.

🔗 Training

- Self-knowledge Distillation: combining multiple outputs from different retrieval modes as reward signal to enhance the performance of single mode(especially for sparse retrieval and multi-vec(colbert) retrieval)
- Efficient Batching: Improve the efficiency when fine-tuning on long text. The small-batch strategy is simple but effective, which also can used to fine-tune large embedding model.
- MCLS: A simple method to improve the performance on long text without fine-tuning. If you have no enough resource to fine-tuning model with long text, the method is useful.

Refer to our [report](#) for more details.

🔗 Acknowledgement

Thanks to the authors of open-sourced datasets, including Miracl, MKQA, NarritiveQA, etc. Thanks to the open-sourced libraries like [Tevatron](#), [Pyserini](#).

🔗 Citation

If you find this repository useful, please consider giving a star :star: and citation

```
@misc{bge-m3,  
  title={BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddi  
  author={Jianlv Chen and Shitao Xiao and Peitian Zhang and Kun Luo and Defu Lian and Zheng L  
  year={2024},  
  eprint={2402.03216},  
  archivePrefix={arXiv},  
  primaryClass={cs.CL}  
}
```